

Introduction to Amazon S3

This introduction to Amazon Simple Storage Service is intended to give you a detailed summary of this web service. After reading this section, you should have a good idea of what it offers and how it can fit in with your business.

Topics

- [Overview of Amazon S3 and This Guide](#)
- [Advantages to Amazon S3](#)
- [Amazon S3 Concepts](#)
- [Features](#)
- [Amazon S3 Application Programming Interfaces \(API\)](#)
- [Paying for Amazon S3](#)
- [Related Services](#)

Overview of Amazon S3 and This Guide

Amazon S3 has a simple web services interface that you can use to store and retrieve any amount of data, at any time, from anywhere on the web.

This guide describes how you send requests to create buckets, store and retrieve your objects, and manage permissions on your resources. The guide also describes access control and the authentication process. Access control defines who can access objects and buckets within Amazon S3, and the type of access (e.g., READ and WRITE). The authentication process verifies the identity of a user who is trying to access Amazon Web Services (AWS).

Advantages to Amazon S3

Amazon S3 is intentionally built with a minimal feature set that focuses on simplicity and robustness. Following are some of advantages of the Amazon S3 service:

- **Create Buckets** – Create and name a bucket that stores data. Buckets are the fundamental container in Amazon S3 for data storage.
- **Store data in Buckets** – Store an infinite amount of data in a bucket. Upload as many objects as you like into an Amazon S3 bucket. Each object can contain up to 5 TB of data. Each object is stored and retrieved using a unique developer-assigned key.
- **Download data** – Download your data or enable others to do so. Download your data any time you like or allow others to do the same.

- **Permissions** – Grant or deny access to others who want to upload or download data into your Amazon S3 bucket. Grant upload and download permissions to three types of users. Authentication mechanisms can help keep data secure from unauthorized access.
- **Standard interfaces** – Use standards-based REST and SOAP interfaces designed to work with any Internet-development toolkit.

Note

SOAP support over HTTP is deprecated, but it is still available over HTTPS. New Amazon S3 features will not be supported for SOAP. We recommend that you use either the REST API or the AWS SDKs.

Amazon S3 Concepts

Topics

- [Buckets](#)
- [Objects](#)
- [Keys](#)
- [Regions](#)
- [Amazon S3 Data Consistency Model](#)

This section describes key concepts and terminology you need to understand to use Amazon S3 effectively. They are presented in the order you will most likely encounter them.

Buckets

A bucket is a container for objects stored in Amazon S3. Every object is contained in a bucket. For example, if the object named `photos/puppy.jpg` is stored in the `johnsmith` bucket, then it is addressable using the URL

```
http://johnsmith.s3.amazonaws.com/photos/puppy.jpg
```

Buckets serve several purposes: they organize the Amazon S3 namespace at the highest level, they identify the account responsible for storage and data transfer charges, they play a role in access control, and they serve as the unit of aggregation for usage reporting.

You can configure buckets so that they are created in a specific region. For more information, see [Buckets and Regions](#). You can also configure a bucket so that every time an object is added to it, Amazon S3 generates a unique version ID and assigns it to the object. For more information, see [Versioning](#).

For more information about buckets, see [Working with Amazon S3 Buckets](#).

Objects

Objects are the fundamental entities stored in Amazon S3. Objects consist of object data and metadata. The data portion is opaque to Amazon S3. The metadata is a set of name-value pairs that describe the object. These include some default metadata, such as the date last modified, and standard HTTP metadata, such as Content-Type. You can also specify custom metadata at the time the object is stored.

An object is uniquely identified within a bucket by a key (name) and a version ID. For more information, see [Keys](#) and [Versioning](#).

Keys

A key is the unique identifier for an object within a bucket. Every object in a bucket has exactly one key. Because the combination of a bucket, key, and version ID uniquely identify each object, Amazon S3 can be thought of as a basic data map between "bucket + key + version" and the object itself. Every object in Amazon S3 can be uniquely addressed through the combination of the web service endpoint, bucket name, key, and optionally, a version. For example, in the URL `http://doc.s3.amazonaws.com/2006-03-01/AmazonS3.wsdl`, "doc" is the name of the bucket and "2006-03-01/AmazonS3.wsdl" is the key.

Regions

You can choose the geographical region where Amazon S3 will store the buckets you create. You might choose a region to optimize latency, minimize costs, or address regulatory requirements. Amazon S3 currently supports the following regions:

- **US East (N. Virginia) region** Uses Amazon S3 servers in Northern Virginia
- **US West (Oregon) region** Uses Amazon S3 servers in Oregon
- **US West (N. California) region** Uses Amazon S3 servers in Northern California
- **EU (Ireland) region** Uses Amazon S3 servers in Ireland
- **EU (Frankfurt) region** Uses Amazon S3 servers in Frankfurt
- **Asia Pacific (Singapore) region** Uses Amazon S3 servers in Singapore
- **Asia Pacific (Tokyo) region** Uses Amazon S3 servers in Tokyo
- **Asia Pacific (Sydney) region** Uses Amazon S3 servers in Sydney
- **Asia Pacific (Seoul) region** Uses Amazon S3 servers in Seoul
- **South America (São Paulo) region** Uses Amazon S3 servers in Sao Paulo

Objects stored in a region never leave the region unless you explicitly transfer them to another region. For example, objects stored in the EU (Ireland) region never leave it. For more information about Amazon S3 regions and endpoints, go to [Regions and Endpoints](#) in the *AWS General Reference*.

Amazon S3 provides read-after-write consistency for PUTS of new objects in your S3 bucket and eventual consistency for overwrite PUTS and DELETES in all regions. The next section provides more information about the Amazon S3 data consistency model

Amazon S3 Data Consistency Model

Updates to a single key are atomic. For example, if you PUT to an existing key, a subsequent read might return the old data or the updated data, but it will never write corrupted or partial data.

Amazon S3 achieves high availability by replicating data across multiple servers within Amazon's data centers. If a PUT request is successful, your data is safely stored. However, information about the changes must replicate across Amazon S3, which can take some time, and so you might observe the following behaviors:

- A process writes a new object to Amazon S3 and immediately lists keys within its bucket. Until the change is fully propagated, the object might not appear in the list.
- A process replaces an existing object and immediately attempts to read it. Until the change is fully propagated, Amazon S3 might return the prior data.
- A process deletes an existing object and immediately attempts to read it. Until the deletion is fully propagated, Amazon S3 might return the deleted data.
- A process deletes an existing object and immediately lists keys within its bucket. Until the deletion is fully propagated, Amazon S3 might list the deleted object.

Note

Amazon S3 does not currently support object locking. If two PUT requests are simultaneously made to the same key, the request with the latest time stamp wins. If this is an issue, you will need to build an object-locking mechanism into your application.

Updates are key-based; there is no way to make atomic updates across keys. For example, you cannot make the update of one key dependent on the update of another key unless you design this functionality into your application.

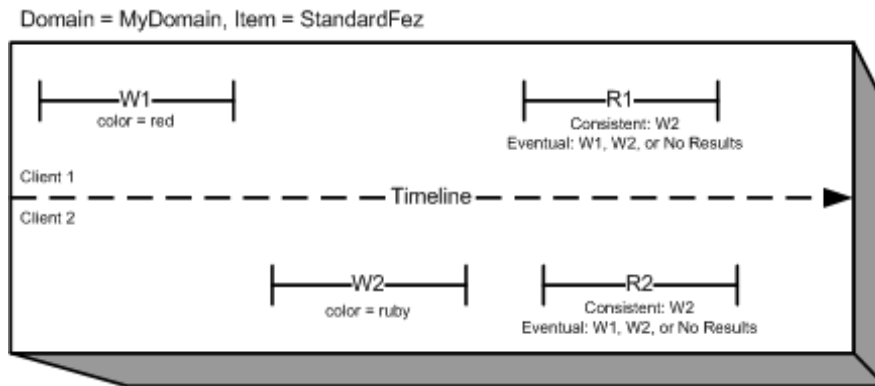
The following table describes the characteristics of eventually consistent read and consistent read.

Eventually Consistent Read	Consistent Read
Stale reads possible	No stale reads
Lowest read latency	Potential higher read latency
Highest read throughput	Potential lower read throughput

Concurrent Applications

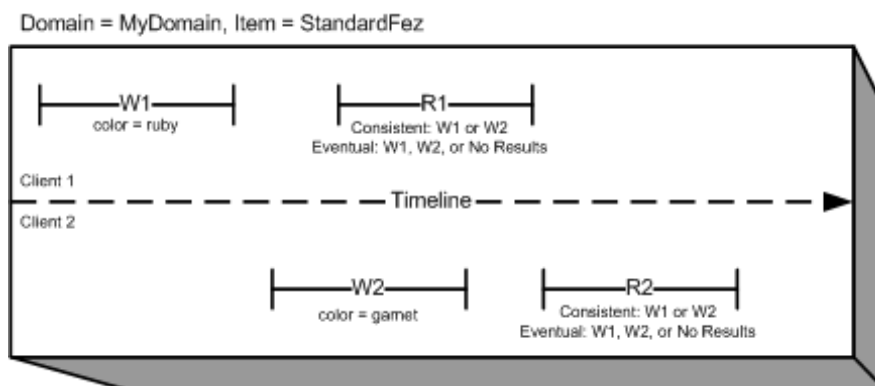
This section provides examples of eventually consistent and consistent read requests when multiple clients are writing to the same items.

In this example, both W1 (write 1) and W2 (write 2) complete before the start of R1 (read 1) and R2 (read 2). For a consistent read, R1 and R2 both return `color = ruby`. For an eventually consistent read, R1 and R2 might return `color = red`, `color = ruby`, or no results, depending on the amount of time that has elapsed.



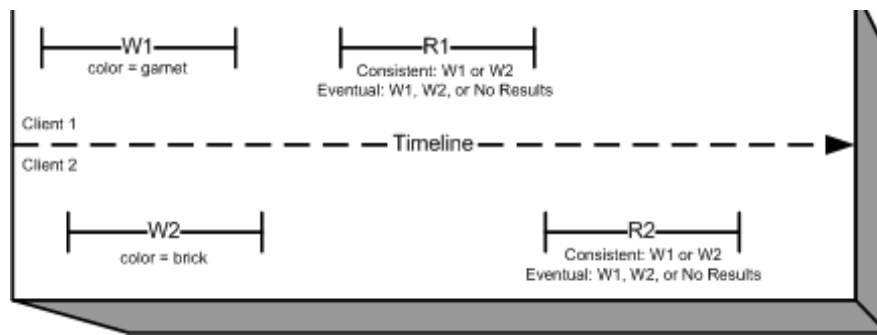
In the next example, W2 does not complete before the start of R1. Therefore, R1 might return `color = ruby` or `color = garnet` for either a consistent read or an eventually consistent read. Also, depending on the amount of time that has elapsed, an eventually consistent read might return no results.

For a consistent read, R2 returns `color = garnet`. For an eventually consistent read, R2 might return `color = ruby`, `color = garnet`, or no results depending on the amount of time that has elapsed.



In the last example, Client 2 performs W2 before Amazon S3 returns a success for W1, so the outcome of the final value is unknown (`color = garnet` or `color = brick`). Any subsequent reads (consistent read or eventually consistent) might return either value. Also, depending on the amount of time that has elapsed, an eventually consistent read might return no results.

Domain = MyDomain, Item = StandardFz



Features

Topics

- [Reduced Redundancy Storage](#)
- [Bucket Policies](#)
- [AWS Identity and Access Management](#)
- [Access Control Lists](#)
- [Versioning](#)
- [Operations](#)

This section describes important Amazon S3 features.

Reduced Redundancy Storage

Customers can store their data using the Amazon S3 Reduced Redundancy Storage (RRS) option. RRS enables customers to reduce their costs by storing non-critical, reproducible data at lower levels of redundancy than Amazon S3 standard storage. RRS provides a cost-effective, highly available solution for distributing or sharing content that is durably stored elsewhere, or for storing thumbnails, transcoded media, or other processed data that can be easily reproduced. The RRS option stores objects on multiple devices across multiple facilities, providing 400 times the durability of a typical disk drive, but does not replicate objects as many times as standard Amazon S3 storage, and thus is even more cost effective.

RRS provides 99.99% durability of objects over a given year. This durability level corresponds to an average expected loss of 0.01% of objects annually.

AWS charges less for using RRS than for standard Amazon S3 storage. For pricing information, see [Amazon S3 Pricing](#).

For more information, see [Storage Classes](#).

Bucket Policies

Bucket policies provide centralized, access control to buckets and objects based on a variety of conditions, including Amazon S3 operations, requesters, resources, and

aspects of the request (e.g., IP address). The policies are expressed in our access policy language and enable centralized management of permissions. The permissions attached to a bucket apply to all of the objects in that bucket.

Individuals as well as companies can use bucket policies. When companies register with Amazon S3 they create an account. Thereafter, the company becomes synonymous with the account. Accounts are financially responsible for the Amazon resources they (and their employees) create. Accounts have the power to grant bucket policy permissions and assign employees permissions based on a variety of conditions. For example, an account could create a policy that gives a user write access:

- To a particular S3 bucket
- From an account's corporate network
- During business hours
- From an account's custom application (as identified by a user agent string)

An account can grant one application limited read and write access, but allow another to create and delete buckets as well. An account could allow several field offices to store their daily reports in a single bucket, allowing each office to write only to a certain set of names (e.g. "Nevada/*" or "Utah/*") and only from the office's IP address range.

Unlike access control lists (described below), which can add (grant) permissions only on individual objects, policies can either add or deny permissions across all (or a subset) of objects within a bucket. With one request an account can set the permissions of any number of objects in a bucket. An account can use wildcards (similar to regular expression operators) on Amazon resource names (ARNs) and other values, so that an account can control access to groups of objects that begin with a common prefix or end with a given extension such as .html.

Only the bucket owner is allowed to associate a policy with a bucket. Policies, written in the access policy language, allow or deny requests based on:

- Amazon S3 bucket operations (such as `PUT ?acl`), and object operations (such as `PUT Object`, or `GET Object`)
- Requester
- Conditions specified in the policy

An account can control access based on specific Amazon S3 operations, such as `GetObject`, `GetObjectVersion`, `DeleteObject`, or `DeleteBucket`.

The conditions can be such things as IP addresses, IP address ranges in CIDR notation, dates, user agents, HTTP referrer and transports (HTTP and HTTPS).

For more information, see [Using Bucket Policies and User Policies](#).

AWS Identity and Access Management

For example, you can use IAM with Amazon S3 to control the type of access a user or group of users has to specific parts of an Amazon S3 bucket your AWS account owns.

For more information about IAM, see the following:

- [Identity and Access Management \(IAM\)](#)
- [Getting Started](#)
- [IAM User Guide](#)

Access Control Lists

For more information, see [Managing Access with ACLs](#)

Versioning

For more information, see [Object Versioning](#).

Operations

Following are the most common operations you'll execute through the API.

Common Operations

- **Create a Bucket** – Create and name your own bucket in which to store your objects.
- **Write an Object** – Store data by creating or overwriting an object. When you write an object, you specify a unique key in the namespace of your bucket. This is also a good time to specify any access control you want on the object.
- **Read an Object** – Read data back. You can download the data via HTTP or BitTorrent.
- **Deleting an Object** – Delete some of your data.
- **Listing Keys** – List the keys contained in one of your buckets. You can filter the key list based on a prefix.

Details on this and all other functionality are described in detail later in this guide.

Amazon S3 Application Programming Interfaces (API)

The Amazon S3 architecture is designed to be programming language-neutral, using our supported interfaces to store and retrieve objects.

Amazon S3 provides a REST and a SOAP interface. They are similar, but there are some differences. For example, in the REST interface, metadata is returned in HTTP

headers. Because we only support HTTP requests of up to 4 KB (not including the body), the amount of metadata you can supply is restricted.

Note

SOAP support over HTTP is deprecated, but it is still available over HTTPS. New Amazon S3 features will not be supported for SOAP. We recommend that you use either the REST API or the AWS SDKs.

The REST Interface

The REST API is an HTTP interface to Amazon S3. Using REST, you use standard HTTP requests to create, fetch, and delete buckets and objects.

You can use any toolkit that supports HTTP to use the REST API. You can even use a browser to fetch objects, as long as they are anonymously readable.

The REST API uses the standard HTTP headers and status codes, so that standard browsers and toolkits work as expected. In some areas, we have added functionality to HTTP (for example, we added headers to support access control). In these cases, we have done our best to add the new functionality in a way that matched the style of standard HTTP usage.

The SOAP Interface

Note

SOAP support over HTTP is deprecated, but it is still available over HTTPS. New Amazon S3 features will not be supported for SOAP. We recommend that you use either the REST API or the AWS SDKs.

The SOAP API provides a SOAP 1.1 interface using document literal encoding. The most common way to use SOAP is to download the WSDL (go to <http://doc.s3.amazonaws.com/2006-03-01/AmazonS3.wsdl>), use a SOAP toolkit such as Apache Axis or Microsoft .NET to create bindings, and then write code that uses the bindings to call Amazon S3.

Paying for Amazon S3

Pricing for Amazon S3 is designed so that you don't have to plan for the storage requirements of your application. Most storage providers force you to purchase a predetermined amount of storage and network transfer capacity: If you exceed that capacity, your service is shut off or you are charged high overage fees. If you do not exceed that capacity, you pay as though you used it all.

Amazon S3 charges you only for what you actually use, with no hidden fees and no overage charges. This gives developers a variable-cost service that can grow with

their business while enjoying the cost advantages of Amazon's infrastructure.

Before storing anything in Amazon S3, you need to register with the service and provide a payment instrument that will be charged at the end of each month. There are no set-up fees to begin using the service. At the end of the month, your payment instrument is automatically charged for that month's usage.

For information about paying for Amazon S3 storage, see [Amazon S3 Pricing](#).

Related Services

Once you load your data into Amazon S3, you can use it with other services that we provide. The following services are the ones you might use most frequently:

- **Amazon Elastic Compute Cloud** – This web service provides virtual compute resources in the cloud. For more information, go to the [Amazon EC2 product details page](#).
- **Amazon Elastic MapReduce** – This web service enables businesses, researchers, data analysts, and developers to easily and cost-effectively process vast amounts of data. It utilizes a hosted Hadoop framework running on the web-scale infrastructure of Amazon EC2 and Amazon S3. For more information, go to the [Amazon Elastic MapReduce product details page](#).
- **AWS Import/Export** – AWS Import/Export enables you to mail a storage device, such as a RAID drive, to Amazon so that we can upload your (terabytes) of data into Amazon S3. For more information, go to the [AWS Import/Export Developer Guide](#).